Benjamin Holderbein

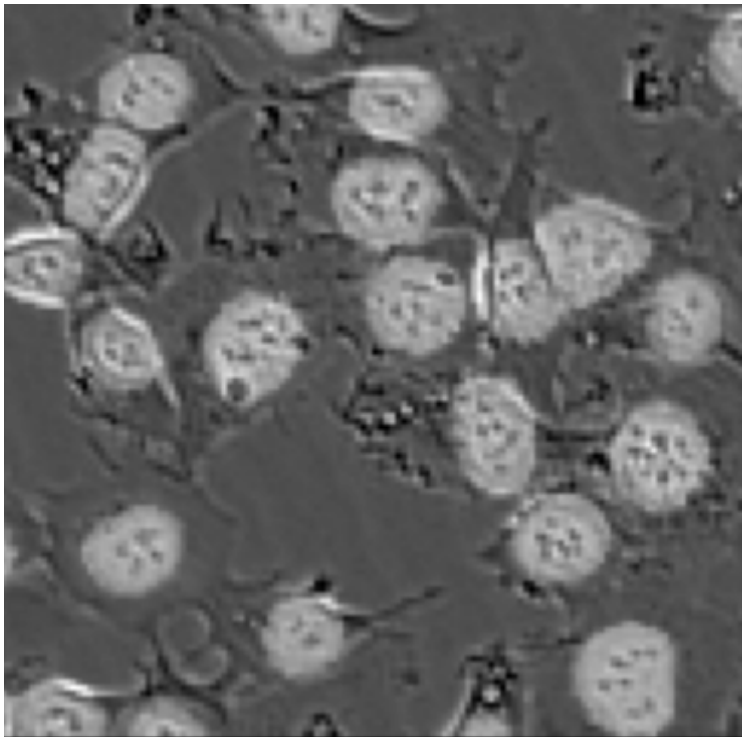
MATH 373

5/12/24

Counting Cells in Microscopy Using a U-Net Convolutional Neural Network

This report documents my process in implementing a U-Net Convolutional Neural Network for Microscopy Image segmentation. The goal of the project was to be able to count the number of cells in a Microscopy image with a mean absolute error (MAE) of less than three for that count. My personal goal was to get that number below one, which I fell short of, with a final MAE of ~1.4.

The training dataset provided for this project consisted of 2,000 images, each with 128x128 resolution. The images were black-and-white microscopy images of cells. The labels were masks for the images that denoted the location and shape of the cells.



Training dataset image with label mask overlaid.

The U-Net architecture makes use of 23 convolutional layers, along with other functions such as ReLU, maxpool, upsample, and matrix stacking. The architecture has two main stages; encoding and decoding. Encoding is the process of pattern recognition where the model finds the cells. During this phase, the model increases its filter count at each layer and lowers the "resolution" of the image. Decoding is the process of mapping those patterns to the image. This is where each progressive layer has fewer filters and more "resolution". The trick that lets the model reclaim its resolution is the matrix stacking process, the model takes data from earlier in the encoding process and stacks it with the current upsampled data to apply its learned patterns to the higher-resolution image. In total, the model has four encoding layers, one base layer, and four decoding layers. My conception of a layer is a set of convolutional layers, each with the same resolution. To learn more about the U-Net architecture you can read this paper.

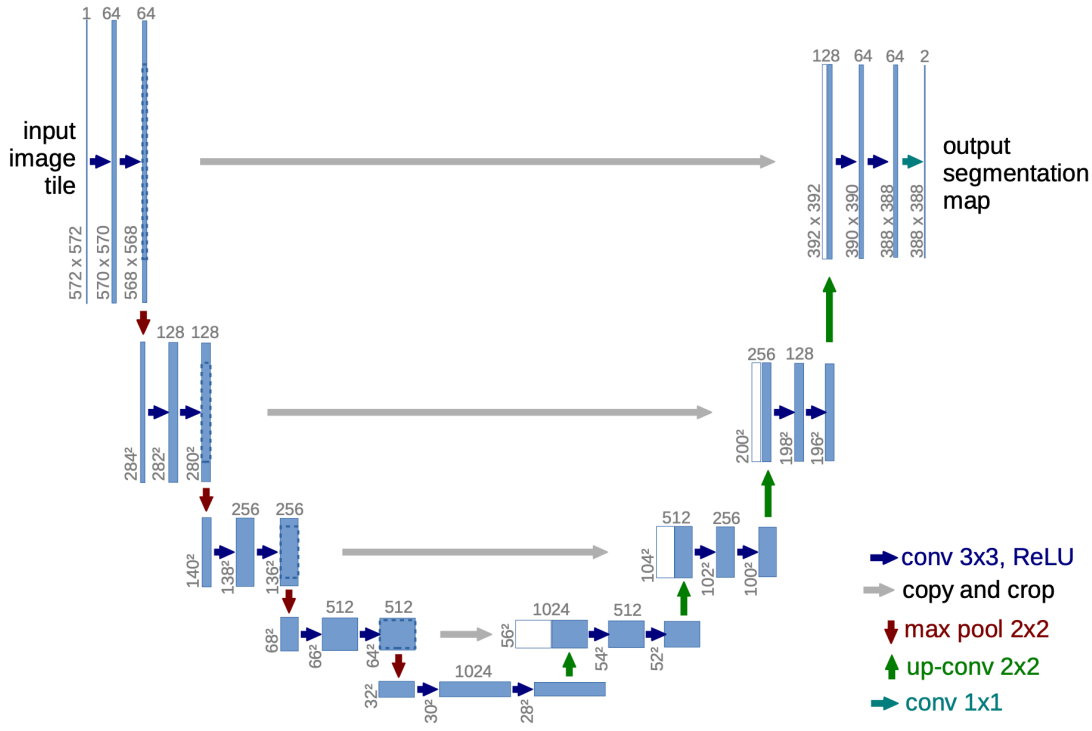
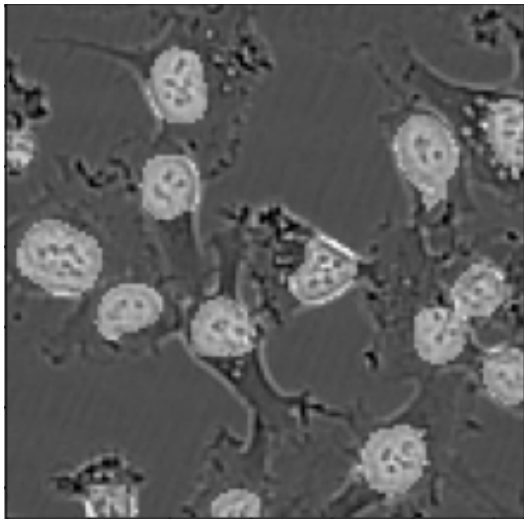


Image from the paper depicting the architecture with 23 convolutional layers.

For submissions, a prediction would leave the model and go into an opencv connected components function. This would analyze the image and count the number of disconnected cells. This was then repeated for all 1000 images in the test dataset and submitted as a CSV.

In implementing this architecture I used three stages. The first was a reduced model to make sure I knew how to implement the architecture. This model (called UNet1 in my code) utilized an encoding layer, a base layer, and a decoding layer, for a total of eight convolutional layers. This model performed respectfully with an MSA of ~9, we need to go further.



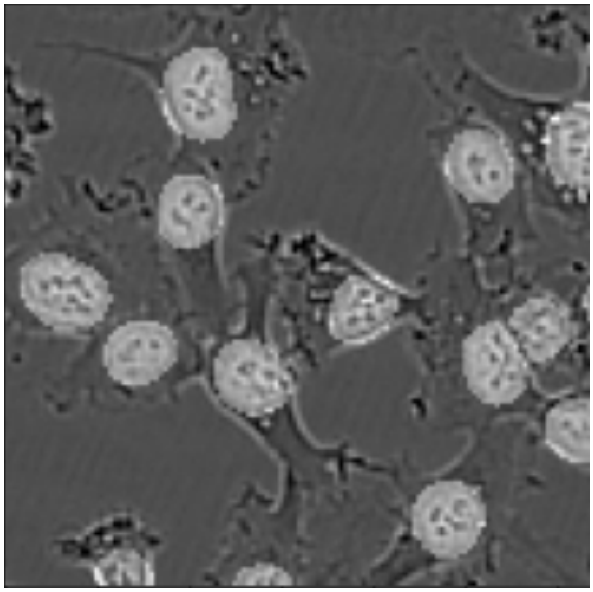
Training dataset image with UNet1 predicted mask overlaid.

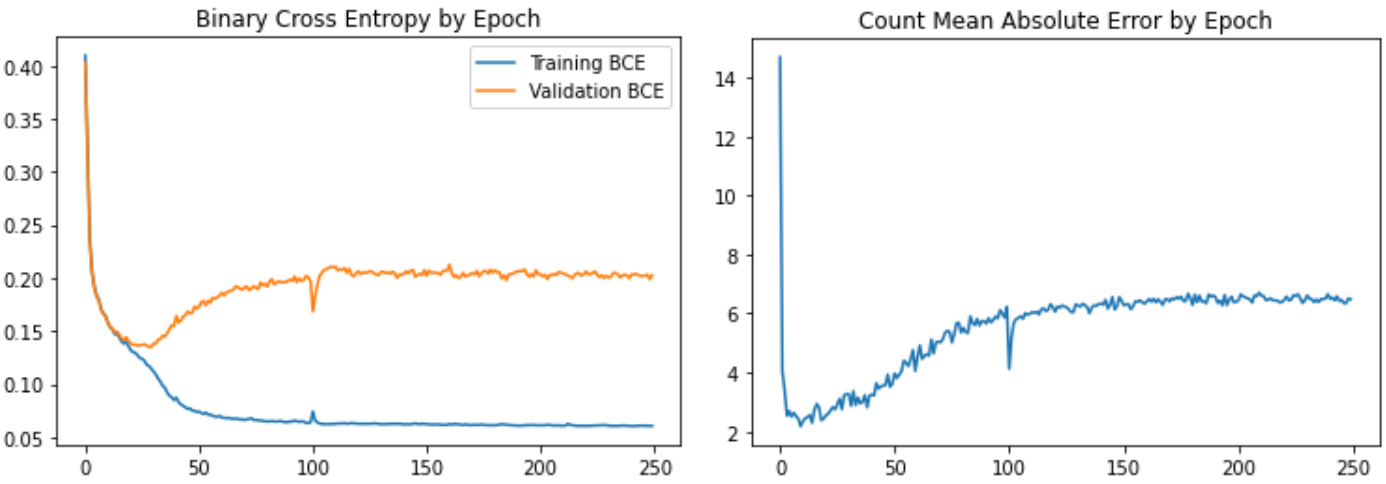
The second stage (UNet 2 in my code) was the full 23 convolutional layer U-Net architecture depicted in the U-Net paper. This version had four encoding layers, one base layer, and four decoding layers. This model took a while to train, it seemed like more epochs were more better. At this point Google Colab stopped playing nice, blocking me from using their GPUs. I knew I needed more compute and I knew just how to get it. Machine learning model training works best on a GPU and I happen to have a gaming computer with a buff 4070 Ti inside. To get this working I had to go through a three-hour process. First I had to get Spyder and the appropriate libraries working on my PC, easy enough. Next, I had to get CUDA to work on

the system. CUDA is Nvidia's machine learning toolkit, what I didn't know at the time was that I needed a very specific version of the library to work with Pytorch. Nothing wanted to tell me this directly but after a few hours, I was able to strangle the information out of ChatGPT. Once I got this working, the sky was the limit when it came to epochs. I tried 20, 50, 100, 250, 500, 1000, I trained my model through the day and the night. Eventually, I found 250 epochs to minimize MAE with a value of ~1.4.

I was kind of upset that I couldn't fix my problems (if only life worked like that). My third and final model (UNet3 in my code) added an encoding and decoding layer, just to see if more layers were better. This version had a total of 28 convolutional layers. Unfortunately, this model performed worse, with a minimized MAE of ~1.7 with 300 epochs.

The loss function used when training was cross-entropy, even though this wasn't the metric we would be evaluated on when submitting. This caused an interesting issue, firstly more epochs would improve MAE even when cross entropy would suggest that I was overfitting. I attempted to implement my own MAE function to evaluate my model by epoch but this also did not match my scores on Kaggle. Even when my MAE metric said I was overfitting, 250 epochs with UNet2 was my best-performing model.





Training plots and sample output with 250 epochs and UNet2.

I also experimented with a bias term. The logic is that the model would show two or more cells touching each other with some frequency, and the connected components would label this as one cell. Maybe we could add some value (even fractional) to the cell counts of every image to account for this. A bias of zero ended up being best, meaning that the concept failed.

At this point, I gave up on my goal of an MAE lower than one. I don't quite know what else to try, or if it is even possible to attain. I had a good time completing this project, I found enjoyment in tweaking my model and submitting a Kaggle submission first thing in the morning

like a little loot box. I learned a lot about optimization and when I learn even more I can return to the project in the future.

P.S. If you have any ideas on how to improve the model please let me know.